

Fine-grained Locking and RW Locks

Fine-Grained Locking

Learning Objectives:

- Be able to explain what lock contention is and why it is a problem.
- Be able to use multiple locks to safely share data structures.
- Be able to use reader-writer locks to accelerate reads of a shared data structure.

Complimentary Reading: [OSTEP](#), Condition Variables (Chapter 30)

Motivation–Review of the HashTable

- How did you solve the concurrency issues?
 - If you tried putting locks around all of the functions, why didn't that work?
- What did you notice about the performance of safe hashtable? Poor scalability!
 - Why?, Lock Contention: the time that threads spend blocked or spinning because they are trying to acquire a lock that is currently held by another thread.
- Is this the best we can do?
 - The table has many independent buckets.
 - The workload accesses random keys, so probably accesses random buckets

Fine-grained locking

- Simple Idea: use a separate lock for each of the logical items in a data structure
- Advantages:
 - Allows more parallelism by reducing lock contention
 - Improves cache locality (a low level hardware concern)
- Disadvantages:
 - More difficult to reason about invariants that span logical items
 - e.g., resizing is hard!
 - New types of concurrency issues can crop up.
- When should we use it: